DEPLOYMENT RUNBOOK · MILESTONE 1

AgentOps Studio Azure Deployment, CI/CD & AI Workflow Execution Runbook

Visual architecture, tab-by-tab VM setup, Linux deployment, verification, CI/CD, AI workflow execution, cost budget, and operations record

Application: AgentOps Studio

Repository: hafiz475/agentops-ai-workflow-platform

Azure resource group: rg-agentops-demo

VM: vm-agentops-demo · Central India

Deployment date: 6 November 2026

Milestone 0 deployment baseline: 15f1212

Milestone 1 application commit: 2ac3732

Prepared as a reproducible deployment record. Generated passwords and the SSH private key are intentionally not included.

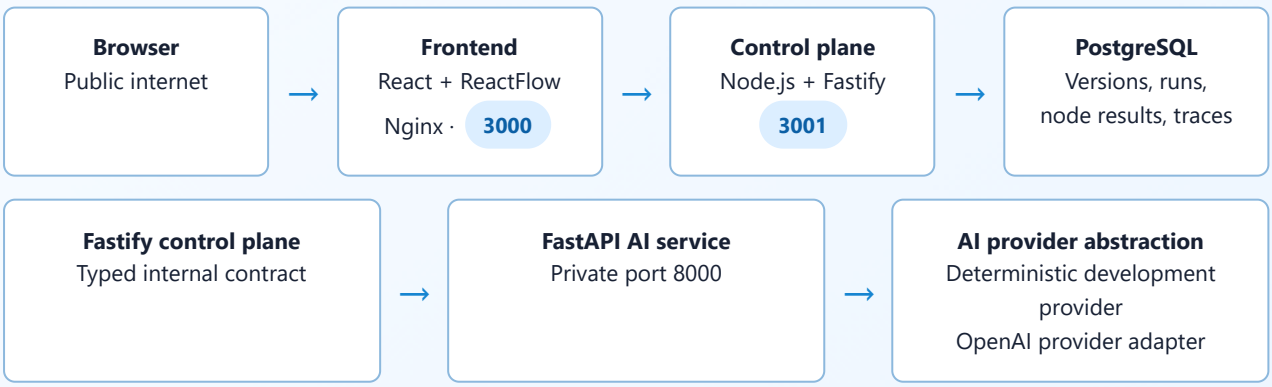
Contents

1. Final outcome and architecture
2. Milestone 1 — AI Workflow Execution
3. Why this deployment design was chosen
4. Create the Azure virtual machine
5. Exact settings on every Azure VM tab
6. Cloud-init and Linux deployment
7. Automatic startup with systemd
8. API health-check failure and permanent fix
9. Deployment verification
10. Create the Azure budget
11. Railway cleanup
12. Automatic CI/CD with GitHub Actions and Azure OIDC
13. Daily operations and maintenance
14. Security, limitations, and shutdown

1. Final outcome and architecture

> **Deployment result:** the complete Milestone 1 application is running successfully on one Azure Linux VM. It now provides functional project and workflow creation, visual editing, immutable versioning, execution, persisted node results, and trace timelines. All five containers start automatically after reboot.

Milestone 1 request and persistence architecture



The browser never calls Python directly. Fastify owns orchestration and calls FastAPI over the private typed contract only for model execution. Redis remains part of the platform infrastructure on the private Compose network. Only ports 3000 and 3001 pass through the Azure NSG.

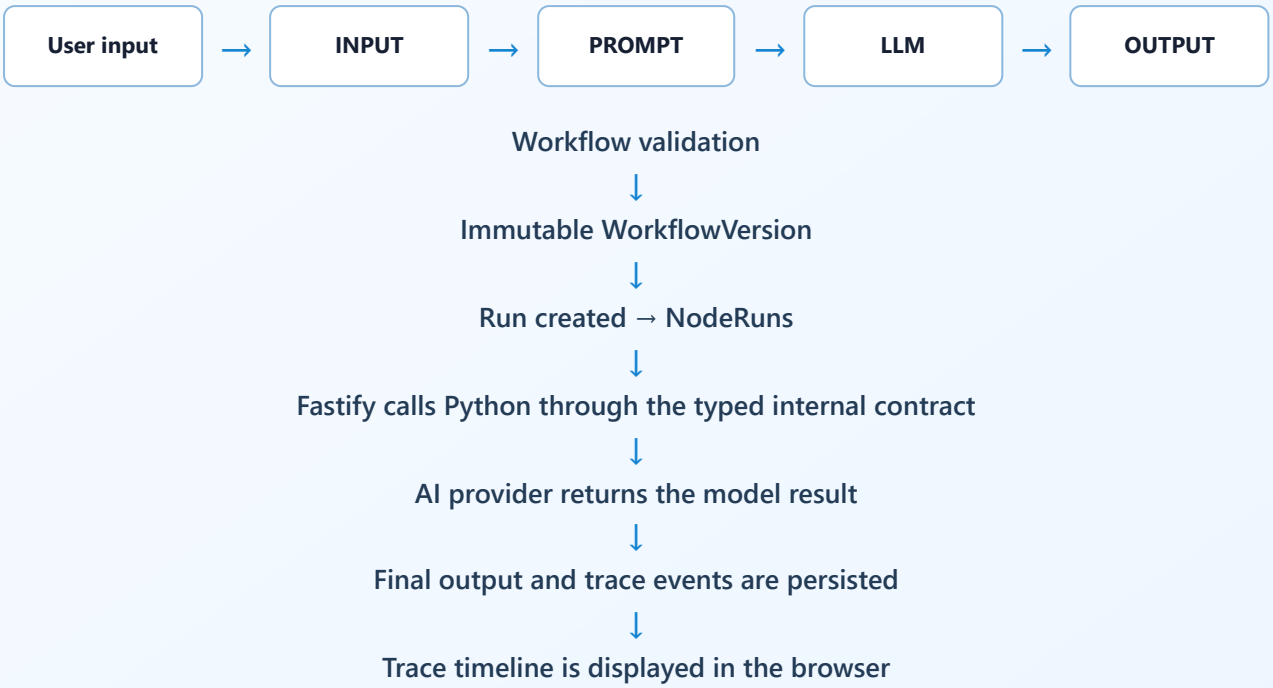
Item	Current value
Resource group	`rg-agentops-demo`
Virtual machine	`vm-agentops-demo`
Region / image	Central India / Ubuntu Server 24.04 LTS, x64 Gen2
VM size	Standard B2ls v2 — 2 vCPUs, 4 GiB RAM
Public IP at verification	`4.247.145.108`
Frontend	http://4.247.145.108:3000/
API health	http://4.247.145.108:3001/health
API readiness	http://4.247.145.108:3001/ready

Container	Milestone 1 purpose	Port exposure
web	React/ReactFlow visual workflow editor served by Nginx	Public TCP 3000 → container 80
api	Fastify control plane: validation, versioning, orchestration, persistence, traces	Public TCP 3001
ai-service	FastAPI model execution and AI provider abstraction	Host 8000; blocked by Azure NSG
postgres	Projects, workflows, versions, runs, node results, and trace data	Private Compose network only, 5432
redis	Platform cache and coordination infrastructure	Private Compose network only, 6379

2. Milestone 1 — AI Workflow Execution

ReactFlow provides the visual editor. A workflow currently contains exactly the supported building blocks needed for the first complete vertical slice: **INPUT**, **PROMPT**, **LLM**, and **OUTPUT** nodes.

Example workflow and execution lifecycle



Before a version can be created, graph validation checks node configuration, edge references, connectivity, reachability, supported node cardinality, prompt variables, and cycles. Saved workflow versions are immutable. A run executes the selected version using deterministic topological ordering. Run state, node-level status, node inputs, node outputs, failures, final output, and trace events are persisted in PostgreSQL.

CONTROL PLANE — Node.js + Fastify

- Projects and workflows
- Immutable versions
- Graph validation
- Run orchestration and deterministic ordering
- Persistence and trace timelines

AI EXECUTION PLANE — Python + FastAPI

- AI provider abstraction
- Model invocation
- Provider-specific integration
- Deterministic development provider
- OpenAI provider adapter

> **Credential-free development:** the deterministic development provider enables reliable local, CI, and demo execution without external AI credentials. The OpenAI adapter is available for real model execution when deployment configuration supplies credentials. Provider credentials must never be embedded in source code or documentation.

Milestone 1 verification record

Verification	Result
GitHub CI / Azure deployment / deployment smoke checks	**Passed**
Node tests	**23 passed**
Python tests	**7 passed**
Formatting, ESLint, TypeScript strict checks	**Passed**
Ruff and mypy	**Passed**
Production builds and Docker Compose validation	**Passed**

3. Why this design was chosen

- **One VM:** Railway's free-plan resource limit could not host the API, AI service, PostgreSQL, and Redis together. One Azure VM could run all five containers and use the available Azure credit.
- **Docker Compose:** the repository already defined all services, health checks, dependency order, volumes, and internal DNS names.
- **Standard B2ls v2:** the originally preferred B2s SKU was unavailable for this subscription in Central India. B2ls v2 was deployable and provided the required 2 vCPUs and 4 GiB RAM.
- **Standard SSD:** enough for the single-VM portfolio/demo deployment while keeping storage cost lower than Premium SSD.
- **Ports 3000 and 3001 only:** users need the frontend and API. PostgreSQL, Redis, and the Python service do not need direct internet access.
- **SSH restricted to one IP:** prevents the whole internet from attempting SSH login.
- **Systemd:** makes the Compose stack return automatically after a VM restart.

4. Create the Azure virtual machine

1. Sign in to the Azure portal.
2. Use the top search bar to search for **Virtual machines**.
3. Open **Virtual machines** under Services, then select **Create** → **Azure virtual machine**.
4. Complete each tab using the settings in the next section. Do not use the ARM64 Ubuntu image; the selected application deployment used x64.

5. Exact settings on every Azure VM tab

Visual memory map — Azure Create VM tabs

1 · Basics
Name, region, Ubuntu x64, B2ls v2, SSH key

2 · Disks
30 GiB Standard SSD, delete with VM

3 · Networking
VNet, subnet, public IP, advanced NSG

4 · Management
Managed identity on; Entra login off

5 · Monitoring
Boot diagnostics on; other extras off

6 · Advanced
Cloud-init in Custom data

7 · Tags
application, environment, milestone

8 · Review
Validate price/settings, create, download key

4.1 Basics

Field	Configured value	Reason
Subscription	Azure subscription 1	Subscription containing the $200 promotional credit.
Resource group	Create new: `rg-agentops-demo`	Keeps every demo resource together for cost tracking and one-step cleanup.
VM name	`vm-agentops-demo`	Clear, application-specific name.
Region	Central India	Chosen region close to the operator; SKU availability was checked manually.
Availability	No infrastructure redundancy required	Single-VM demo; avoids additional resources and cost.
Security type	Trusted launch virtual machines	Enables Secure Boot and vTPM.
Image	Ubuntu Server 24.04 LTS, x64 Gen2	Long-term-support Linux with Docker compatibility.
Architecture	x64	Matches the selected container/runtime path.
Size	Standard B2ls v2	B2s was unavailable; this SKU supplied 2 vCPUs and 4 GiB RAM.
Spot discount	Off / On-demand	Avoids eviction during the demo.
Authentication	SSH public key	Safer than password authentication.
Username	`azureuser`	Linux administrator account used for SSH.
SSH key source	Generate new key pair	Azure generated the public/private pair.
Key type / name	RSA / `agentops-demo-key`	Compatible OpenSSH key.
Public inbound ports	None	Custom, restricted rules were created on the Networking tab.

> **SSH private key:** Azure downloaded it once during VM creation. Store it securely. Never upload it to GitHub, paste it into cloud-init, or include it in documentation.

4.2 Disks

Field	Configured value
Encryption at host	Off (not registered for the subscription)
OS disk size	Image default — 30 GiB
OS disk type	Standard SSD (StandardSSD_LRS)
Delete with VM	Enabled
Ultra Disk compatibility	Disabled
Key management	Platform-managed key
Data disks	None
Ephemeral OS disk	None

4.3 Networking

Field	Configured value
Virtual network	New `vnet-centralindia-1`
Subnet	New `snet-centralindia-1`, `172.16.0.0/24`
Public IP	New `vm-agentops-demo-ip`
NIC network security group	Advanced → new `vm-agentops-demo-nsg`
Delete public IP and NIC with VM	Enabled
Accelerated networking	Enabled/required by selected size
Load balancing	None

Visual memory map — the two inbound rules we created

Rule 1 · Restricted SSH

TCP 22

Source: `115.97.8.157/32`
Priority: 900
Only your public IP

Rule 2 · Public application

TCP 3000, 3001

Source: Any
Priority: 1010
Frontend + API only

Never open: 8000, 5432, or 6379. If your home IP changes, update only the SSH source /32.

Inbound rule 1 — restricted SSH

Setting	Value
Source	My IP address: `115.97.8.157/32`
Source port range	`1-65535` (the portal rejected `*` during this session)
Destination / service	Any / SSH
Destination port / protocol	22 / TCP
Action / priority	Allow / 900
Name	`default-allow-ssh`

Inbound rule 2 — public frontend and API

Setting	Value
Source / source ports	Any / `1-65535`
Destination / service	Any / Custom
Destination ports / protocol	`3000,3001` / TCP
Action / priority	Allow / 1010
Name / description	`allow-agentops-web-api` / Public AgentOps frontend and API

> **Intentionally not opened:** 8000 (AI service), 5432 (PostgreSQL), and 6379 (Redis). They remain unavailable from the internet.

4.4 Management

Setting	Value
System-assigned managed identity	On
Microsoft Entra ID login	Off
Auto-shutdown	Off during creation
Periodic assessment	Off
Patch orchestration	Image default
Hibernation	Off
Backup / Site Recovery	Off

4.5 Monitoring

Setting	Value
Recommended alert rules	Off
Boot diagnostics	Enabled with managed storage account
OS guest diagnostics	Off
Application health monitoring	Off

4.6 Advanced

Setting	Value
Extensions	None
VM applications	None
Custom data	Cloud-init deployment script (see Section 5)
NVMe optimization	Off / unsupported for selected size
Install SQL Server	Off
Capacity reservation	None

4.7 Tags

Applied to all selectable resources:

Name	Value
`application`	`agentops-studio`
`environment`	`demo`
`milestone`	`0`

4.8 Review + create

1. Confirm validation passes and the size is **Standard B2ls v2**.
2. Review the displayed compute estimate: **$0.0448 USD/hour**, approximately **$32.70/month** for the VM. Disk, public IP, monitoring, tax, and bandwidth can add cost.
3. Select **Create**, then download the generated private key when prompted.
4. Wait for **Your deployment is complete**, then select **Go to resource**.

6. Cloud-init and Linux deployment

The VM bootstrap installed Git, Docker Engine, and Docker Compose v2; cloned the `main` branch; created a protected `.env` with a generated PostgreSQL password; built the containers; and started the stack.

> **Secret handling:** the real generated PostgreSQL password is stored only on the VM and is omitted here. The following is the sanitized, reproducible form of the deployment procedure.

```
#!/usr/bin/env bash
set -euo pipefail

apt-get update
apt-get install -y ca-certificates curl git openssl
curl -fsSL https://get.docker.com | sh
usermod -aG docker azureuser

git clone --branch main \
  https://github.com/hafiz475/agentops-ai-workflow-platform.git \
  /opt/agentops-ai-workflow-platform
cd /opt/agentops-ai-workflow-platform

POSTGRES_PASSWORD="$(openssl rand -hex 24)"
install -m 600 /dev/null .env
printf 'POSTGRES_USER=agentops\nPOSTGRES_DB=agentops\nPOSTGRES_PASSWORD=%s\n' \
  "$POSTGRES_PASSWORD" > .env

docker compose up -d --build
docker compose ps
```

The equivalent custom-data field begins with `#cloud-config` and invokes these commands under `runcmd`. Azure processed the custom data with cloud-init during first boot.

Ways to run Linux commands later

Azure portal: VM → Operations → Run command → RunShellScript. This was used to inspect and repair the deployment without opening an unrestricted SSH rule.

SSH from the allowed IP:

```
ssh -i .\agentops-demo-key.pem azureuser@4.247.145.108
```

If the home/public IP changes, update the SSH NSG source to the new exact `/32` address before attempting SSH. Never change it to Any or `0.0.0.0/0`.

7. Automatic startup with systemd

A service unit was created at `/etc/systemd/system/agentops-compose.service`:

```
[Unit]
Description=AgentOps Studio Docker Compose
Requires=docker.service
After=docker.service network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/agentops-ai-workflow-platform
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose stop
TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable agentops-compose.service
sudo systemctl start agentops-compose.service
sudo systemctl status agentops-compose.service --no-pager
```

Why: Docker restarts with the VM, and systemd reliably recreates/starts the Compose application in dependency order.

8. API health-check failure and permanent fix

Observed symptoms

- PostgreSQL, Redis, and the AI service were healthy.
- The API answered successfully from the VM, but Docker marked it unhealthy.
- The web container did not start because it depends on a healthy API.
- The systemd unit failed because `docker compose up -d` reported that dependency failure.

Diagnosis commands

```
cd /opt/agentops-ai-workflow-platform
sudo docker compose ps
sudo docker compose logs --tail=100 api
sudo docker compose exec api wget -qO- http://localhost:3001/ready
sudo docker compose exec api wget -qO- http://127.0.0.1:3001/ready
sudo journalctl -u agentops-compose.service -n 100 --no-pager
```

`localhost` resolved to IPv6 inside the API container while Fastify was reachable over IPv4. Therefore the readiness request worked at `127.0.0.1` but failed at `localhost`.

Permanent repository fix

The API health check in `docker-compose.yml` was changed from:

```
http://localhost:3001/ready
```

to:

```
http://127.0.0.1:3001/ready
```

The change was committed and pushed to GitHub:

```
15f1212 fix: use IPv4 for API container healthcheck
```

Update the VM

```
cd /opt/agentops-ai-workflow-platform
sudo git pull --ff-only origin main
sudo docker compose up -d --build
sudo systemctl daemon-reload
sudo systemctl restart agentops-compose.service
sudo docker compose ps
```

The VM fast-forwarded from `ffbeac5` to `15f1212`. The temporary override used during diagnosis was removed, leaving the native Compose file as the source of truth.

9. Deployment verification

On the VM

```
cd /opt/agentops-ai-workflow-platform
sudo docker compose ps
sudo systemctl is-enabled agentops-compose.service
sudo systemctl is-active agentops-compose.service
curl -fsS http://127.0.0.1:3000/ >/dev/null && echo "frontend: ok"
curl -fsS http://127.0.0.1:3001/health
curl -fsS http://127.0.0.1:3001/ready
curl -fsS http://127.0.0.1:8000/health
```

Expected results:

- All five containers are Up; API, AI service, PostgreSQL, and Redis report healthy.
- The service reports `enabled` and `active`.
- API health and readiness return `{"status":"ok",...}`.

From a browser

- `http://4.247.145.108:3000/` — dashboard loads.
- `http://4.247.145.108:3001/health` — API health is OK.
- `http://4.247.145.108:3001/ready` — dependencies are ready.

> **Verified final state:** all containers healthy, systemd enabled and active, frontend reachable, and both API probes returning OK.

10. Create the Azure budget

1. In Azure, open **Resource groups**.
2. Open `rg-agentops-demo`.
3. Open **Cost Management** → **Budgets**.
4. Select **Add/Create budget**.
5. Set the budget name to `agentops-demo-budget`, reset period to **Monthly**, and amount to **$150 USD**.
6. Add an **Actual cost** alert at **75%**.
7. Add a **Forecasted cost** alert at **90%**.
8. Leave Action group as **None** and add the deployment-owner email once. If Azure shows “Unique email address is required,” delete the invalid/duplicate row, enter the address in the empty row, and press Tab once.
9. Save the budget and confirm it appears in the resource-group budget list.

> **Important:** an Azure budget sends notifications; it does not automatically stop the VM or prevent charges. Stop/deallocate or delete resources when they are not needed.

11. Railway cleanup

After Azure was fully verified, the unused Railway project was deleted. This removed the Railway PostgreSQL and Redis instances, their volumes, and the staged API service. It did not affect Azure because the Azure VM uses its own local Compose volumes and services.

1. Open the Railway project `fantastic-bravery`.
2. Open Project Settings → Danger Zone.
3. Select Delete Project and confirm the exact project name.
4. Verify that the old project and resources no longer appear.

12. Automatic CI/CD with GitHub Actions and Azure OIDC

> **Verified Milestone 1 result:** a push to `main` runs CI first. Only after CI succeeds does GitHub authenticate to Azure without a stored password, ask Azure Run Command to deploy the exact tested commit, rebuild the containers, apply database migrations during API startup, restart the Milestone 1 application, and run health checks.

Visual memory map — automatic delivery path

1 **Push to main** GitHub receives the new commit.	**2** **CI quality gates** Formatting, ESLint, TypeScript, Node/Python tests, builds, Compose validation.	**3** **workflow_run** Deploy starts only when CI conclusion is success.
4 **OIDC login** GitHub exchanges a short-lived token with Microsoft Entra ID.	**5** **Azure Run Command** Executes inside `vm-agentops-demo`; no SSH key is stored in GitHub.	**6** **Deploy + verify** Fast-forward exact SHA, build, restart, check frontend and API readiness.

12.1 Why OIDC was used instead of a self-hosted runner

The repository is public. A self-hosted runner attached to a public repository increases the risk that untrusted workflow code reaches the VM. The final design uses a GitHub-hosted runner plus Azure OpenID Connect. It stores no Azure client secret and no SSH private key in GitHub.

12.2 Azure identity created in Cloud Shell

Cloud Shell was first registered under Subscription → Resource providers → `Microsoft.CloudShell`. An app registration and service principal were then created for `github-agentops-deploy`.

```
az ad app list \
  --display-name "github-agentops-deploy" \
  --query '[0].{clientId:appId,objectId:id}' \
  --output json

az ad sp list \
  --filter "appId eq '<AZURE_CLIENT_ID>'" \
  --query '[0].{servicePrincipalId:id,clientId:appId}' \
  --output json
```

The federated credential was created with the issuer `https://token.actions.githubusercontent.com`, audience `api://AzureADTokenExchange`, and the exact subject printed by the GitHub Actions login log:

```
az ad app federated-credential create \
  --id "<APPLICATION_OBJECT_ID>" \
  --parameters '{
    "name": "github-agentops-azure-production",
    "issuer": "https://token.actions.githubusercontent.com",
    "subject": "repo:hafiz475@87468194/agentops-ai-workflow-platform@1447357531:environment:azure-
production",
    "audiences": ["api://AzureADTokenExchange"]
  }'
```

> **Important distinction:** GitHub's `AZURE_CLIENT_ID` must contain the application/client ID, not the application object ID or service-principal object ID. Registration tokens and private keys must never be copied into this guide.

12.3 Least-privilege Azure role

The service principal received **Virtual Machine Contributor** only on `vm-agentops-demo`, not on the subscription or entire resource group.

```
VM_ID="$(az vm show --resource-group rg-agentops-demo \
  --name vm-agentops-demo --query id -o tsv)"

az role assignment create \
  --assignee-object-id "<SERVICE_PRINCIPAL_OBJECT_ID>" \
  --assignee-principal-type ServicePrincipal \
  --role "Virtual Machine Contributor" \
  --scope "$VM_ID"
```

12.4 GitHub repository configuration

Repository → Settings → Secrets and variables → Actions:

Type	Name	Purpose
Secret	`AZURE_CLIENT_ID`	Microsoft Entra application/client ID.
Secret	`AZURE_TENANT_ID`	Directory/tenant containing the app registration.
Secret	`AZURE_SUBSCRIPTION_ID`	Subscription containing the VM.
Variable	`AZURE_VM_DEPLOY_ENABLED=true`	Emergency enable/disable gate for deployments.

A GitHub environment named `azure-production` matches the federated credential subject.

12.5 Workflow behavior

The workflow is stored at `.github/workflows/deploy-azure.yml`. It uses `id-token: write`, runs on GitHub-hosted Ubuntu, logs in with `azure/login@v2`, and invokes Azure CLI Run Command against the VM.

```
on:
  workflow_run:
    workflows: [CI]
    types: [completed]
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  id-token: write

jobs:
  deploy:
    if: vars.AZURE_VM_DEPLOY_ENABLED == 'true' &&
        (github.event_name == 'workflow_dispatch' ||
         github.event.workflow_run.conclusion == 'success')
    runs-on: ubuntu-latest
    environment: azure-production
```

12.6 Safe deployment script on the VM

`scripts/deploy-azure-vm.sh` rejects an invalid SHA, refuses a dirty production checkout, verifies the SHA belongs to `origin/main`, fast-forwards to that exact tested commit, rebuilds Compose, starts services, and waits for both endpoints:

```
http://127.0.0.1:3001/ready
http://127.0.0.1:3000/
```

If verification fails, it prints container status and recent API/web logs and exits non-zero, making the GitHub deployment red.

12.7 Failure encountered and exact fix

The first OIDC login failed with `AADSTS700016`. Two values were corrected:

1. The GitHub secret was updated to the true application/client ID.
2. The federated subject was replaced with the exact subject claim shown in the failed GitHub login log, including GitHub's repository owner and repository numeric identifiers.

After these corrections, the manual deployment passed. A later push passed CI and automatically triggered the Azure deployment, proving the complete chain.

12.8 How to operate the pipeline

- **Normal use:** push or merge to `main`; no manual Azure action is needed.
- **Manual retry:** Actions → Deploy Azure VM → Run workflow.
- **Emergency pause:** set `AZURE_VM_DEPLOY_ENABLED` to `false`.
- **Verify:** confirm both CI and Deploy Azure VM are green, then open the frontend, `/health`, and `/ready`.

12.9 Milestone 2 deployment verification

Milestone 2 adds the prompt registry, immutable PromptVersions, evaluation datasets and cases, deterministic evaluators, aggregate metrics, and prompt-version comparison. No new public ports or Azure resources are required: the database migration is applied by API startup and the same web, API, AI service, PostgreSQL, and Redis containers are rebuilt by the verified OIDC pipeline.

1. Push the Milestone 2 commit to `main`.
2. Confirm the **CI** workflow completes successfully.
3. Confirm **Deploy Azure VM** runs for the exact successful CI commit.
4. Open the Prompts page and create a prompt plus an immutable version.
5. Select that PromptVersion in a workflow, create a WorkflowVersion, and run it.
6. Create an evaluation dataset with multiple cases, launch an evaluation, and wait for a terminal status.
7. Verify per-case trace links, pass rate, latency percentiles, token totals, and comparison deltas.

Evaluation jobs currently execute inside the API process. PostgreSQL persists their states and results, but interrupted queued/running work is not automatically recovered until durable workers are introduced in a later milestone.

13. Daily operations and maintenance

Check status

```
cd /opt/agentops-ai-workflow-platform
sudo docker compose ps
sudo systemctl status agentops-compose.service --no-pager
```

View logs

```
sudo docker compose logs --tail=200
sudo docker compose logs -f api
sudo journalctl -u agentops-compose.service -n 200 --no-pager
```

Deploy a new commit from main

```
cd /opt/agentops-ai-workflow-platform
sudo git pull --ff-only origin main
sudo docker compose up -d --build
sudo docker compose ps
```

Restart application

```
sudo systemctl restart agentops-compose.service
```

Stop or start the VM to control cost

Use Azure VM Overview → **Stop** and wait until the status is **Stopped (deallocated)**. A shutdown performed only inside Linux may leave compute allocated. Start it again from the same Azure page.

The public IP can change after deallocation unless it is configured as static. Re-check the VM Overview page before using the URLs.

14. Security, limitations, and final checklist

Security controls in place

- SSH key authentication; no password login.
- SSH restricted to `115.97.8.157/32` at deployment time.
- Only ports 3000 and 3001 are publicly allowed for the app.
- Database, Redis, and AI execution ports are not allowed by the Azure NSG.

- PostgreSQL password generated on the VM and protected with file mode 600.
- Trusted Launch, Secure Boot, and vTPM enabled.
- OS disk, NIC, and public IP configured for cleanup with VM deletion.

Current limitations

- The deployment currently uses HTTP and a raw public IP; HTTPS and a domain are not configured.
- Application authentication and authorization are still planned.
- Workflow execution runs asynchronously inside the Fastify API process instead of a durable worker queue.
- Only INPUT, PROMPT, LLM, and OUTPUT node types are currently supported.
- RAG, tools, conditions, human approval, and evaluations are not yet implemented.
- The single-VM portfolio/demo architecture is not highly available.
- PostgreSQL and Redis use Docker volumes on the VM.
- Independent production backup and recovery are not yet configured.
- Provider credentials are deployment configuration and must never appear in documentation.
- The Azure promotional credit expires; the budget only alerts.

Final checklist

Status	Check
Complete	VM running in Central India
Complete	Five-container Compose stack healthy
Complete	Frontend and API reachable publicly
Complete	API readiness verifies PostgreSQL, Redis, and AI service
Complete	Systemd enabled and active
Complete	IPv4 health-check fix committed and pushed
Complete	Azure budget saved
Complete	Unused Railway project deleted
Complete	GitHub OIDC identity and VM-scoped role configured
Complete	Milestone 2 uses the same CI-gated automatic Azure deployment pipeline

---

End of runbook · AgentOps Studio Milestone 2 · Azure deployment, GitHub OIDC CI/CD, and AI workflow execution